barrier/linear

An AgentMPI collective scaling analysis

Abstract

The linear barrier is the pattern every hand-written "wait for all the workers" harness actually implements: each non-root rank notifies rank 0 and blocks, rank 0 collects $p-1$ notifications and then releases $p-1$ ranks. It costs $2(p-1)$ messages, and the sweep confirms that at all twenty-one measured sizes — 14 at $p=8$, 62 at $p=32$ — with the traffic split exactly as the algorithm dictates: one send from each leaf, $p-1$ from the root. It is the cheapest barrier in the sweep by message count and the most expensive by latency, because the root's $p-1$ receives and $p-1$ sends are serialised, which is what the closed form's $2(p-1)$ rounds means. Two things are wrong with how the trace reports it and both are visible only across the family. **The `rounds` field is 0 in every `coll.barrier` record in the repository**, because `barrier()` is the only collective in `algorithms.py` that never assigns `tr.stats.rounds`; and the root's serialisation, which the closed form prices at $2(p-1)$, is therefore invisible in the metrics even though it is plainly visible in the viewer, where rank 0's lane is the only one drawn with bars. The deeper point is that for agent ranks a barrier's cost is set by its straggler, and that requires the real-agent runs: in `real-sw-p8-full` one integration barrier held eight ranks for between 0 and 215.6 s each.

1 What this family is

The `barrier` collective under the `linear` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [ok] All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [note] Wall times span 0.013–0.125 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

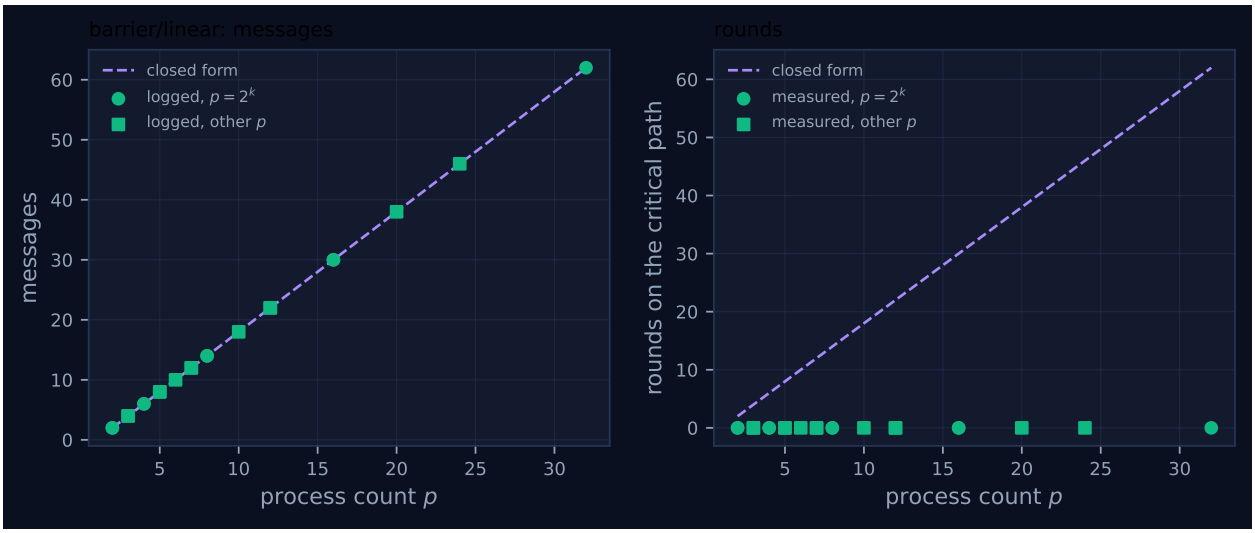

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	0	2	0.014	✓	✓
2	`mb-smoke`	2	2	2	0	2	0.013	✓	✓
3	`mb-free`	4	4	4	0	4	0.027	✓	✓
3	`mb-smoke`	4	4	4	0	4	0.014	✓	✓
4	`mb-free`	6	6	6	0	6	0.016	✓	✓
4	`mb-smoke`	6	6	6	0	6	0.016	✓	✓
5	`mb-free`	8	8	8	0	8	0.033	✓	✓
5	`mb-smoke`	8	8	8	0	8	0.027	✓	✓
6	`mb-free`	10	10	10	0	10	0.019	✓	✓
7	`mb-free`	12	12	12	0	12	0.046	✓	✓
7	`mb-smoke`	12	12	12	0	12	0.021	✓	✓
8	`mb-free`	14	14	14	0	14	0.037	✓	✓
8	`mb-smoke`	14	14	14	0	14	0.024	✓	✓
10	`mb-free`	18	18	18	0	18	0.038	✓	✓
12	`mb-free`	22	22	22	0	22	0.041	✓	✓
12	`mb-smoke`	22	22	22	0	22	0.035	✓	✓
16	`mb-free`	30	30	30	0	30	0.063	✓	✓
16	`mb-smoke`	30	30	30	0	30	0.090	✓	✓
20	`mb-free`	38	38	38	0	38	0.093	✓	✓
24	`mb-free`	46	46	46	0	46	0.125	✓	✓
32	`mb-free`	62	62	62	0	62	0.102	✓	✓

4 How the algorithm scales

The implementation branches on rank. A non-root sends one token to rank 0 under the "up" tag and then blocks on a receive from rank 0 under the "down" tag: two messages of involvement, one of them a send. Rank 0 executes `for _ in range(p - 1)` wildcard receives on the up tag, and

then `for d in range(1, p)` sends on the down tag. So the message count is $(p-1)$ up plus $(p-1)$ down, and `FORMULAS["barrier","linear"]` records $2(p-1)$ messages with $2(p-1)$ rounds and $2(p-1)$ tokens of volume, the payload being the integer 1.

The $2(p-1)$ rounds deserve unpacking, because it is the number that makes this algorithm bad and the sweep's `rounds` column does not show it. It is not a count of phases — there are two phases, fan-in and fan-out. It is the critical path measured in message costs: the root must complete its $p-1$ receives before it issues its first send, and issues its $p-1$ sends one at a time, so $2(p-1)$ messages lie in sequence on the path from the first arrival to the last release. The docstring puts it exactly: "$2(p-1)$ messages and $2(p-1)\alpha$ latency *at the root*, but only 2α at the leaves". A leaf's experience of this barrier is two messages; the root's is $2(p-1)$, and since no rank leaves until the root has sent to it, the root's experience is the barrier's.

The message panel of Figure 1 is the straight line $2(p-1)$ and matches at every one of the twenty-one rows: 4 at $p=3$, 14 at $p=8$, 22 at $p=12$, 38 at $p=20$, 62 at $p=32$. There is nothing to explain about its shape — no logarithm, no staircase, no code path that depends on p — which is precisely why it is the wrong algorithm at scale: the cost is linear in p where the alternative is logarithmic.

The round panel is a defect rather than a measurement. The measured round count is 0 at all twenty-one sizes against a closed form rising from 2 to 62. `barrier()` computes `rounds = 2` for this algorithm and returns it inside `BarrierResult`, but never assigns `tr.stats.rounds`, so the emitted `coll.barrier` event carries the `CollStats` default of 0. It is the only collective in the module with that omission — twenty-five other algorithm branches assign it explicitly — and the consequence reaches every barrier record in the repository: all forty-two runs across the two barrier sweep families, and all four barriers in `real-sw-p8-full`, where `metrics.json` duly reports `"rounds_agree": false`. It survived because the benchmark that generated these runs reads both `rounds_measured` and `rounds_model` and then tests only `messages_match` and `fold_depth_match`; rounds are recorded and never compared. This family table is the first place in the repository where the two sit in adjacent columns.

Note that even a fixed implementation would not make this column agree, and the discrepancy is worth naming rather than papering over. The implementation's notion of "rounds" for this algorithm is 2 — two phases — while the closed form's is $2(p-1)$ — the root's serialised path. Both are defensible quantities and they are not the same quantity. The module is not consistent about which it records: `bcast/flat`, a structurally identical fan-out, sets `tr.stats.rounds = 1 if r != root else p - 1`, choosing the serialised reading and making it rank-dependent, which is the honest choice and the one that would have matched here.

What the trace does show, unambiguously, is the asymmetry. At $p=8$ rank 0's `coll.barrier` record reports `messages_sent: 7` and every other rank reports 1; at $p=32$ it is 31 against 1. The viewer screenshot for `mb-free__coll-barrier-linear-32` makes the serialisation visual: rank 0's lane is the only one containing thick green bars — two of them, at roughly 0.035–0.040 s and 0.078–0.085 s, which are the fan-in and the fan-out — while all thirty-one other lanes carry exactly two thin ticks. The viewer's encoding is that bars have extent and ticks are instants, so the picture is saying that only one rank in this collective did anything with duration. Put next to `mb-free__coll-barrier-dissemination-32`, where all thirty-two lanes are identical, it is the clearest single image of what a root costs. Quantitatively, rank 0's 31 receives at $p=32$ account for 0.348 s of the 0.82 rank-seconds of receive-blocking in the whole collective — 42 % of it in one rank out of thirty-two.

5 Powers of two and everything else

This algorithm has no power-of-two structure to break. Both loops are over `range(p)` or `range(1, p)`, there is no bit arithmetic anywhere in the branch, and the wildcard receive at the root does not care which order the leaves arrive in. The family view confirms the prediction that follows: the squares in Figure 1 lie on $2(p-1)$ exactly as the circles do — 4, 8, 10, 12, 18, 22, 38 and 46 at $p = 3, 5, 6, 7, 10, 12, 20, 24$ — and there are no red crosses, so the self-reported count equalled the logged traffic at all twenty-one sizes. The eight sizes measured in both `mb-free` and `mb-smoke` agree on every integer count and differ only in wall time (0.037 s against 0.024 s at $p = 8$).

That makes this family a useful control. When a sibling family shows a non-power-of-two anomaly, the question is always whether the anomaly is in the algorithm or in the measurement apparatus; a family with no remainder path and no disagreement at any size establishes that the apparatus is not the problem. The one anomaly this family does show — the zero round count — is present at powers of two and elsewhere alike, which is itself the diagnostic that it is an instrumentation omission rather than a code path.

6 When to choose this algorithm

Not for agent ranks, and the closed forms make that a one-line argument. At $p = 32$ the critical path is $2(p-1) = 62$ sequential message costs against $\lceil \log_2 p \rceil = 5$ for dissemination and 2 for the central barrier. Under the default cost parameters, `cost.predict("barrier", ..., p=32, n=4096)` gives 31.93 s for linear against 2.58 s for dissemination and 1.03 s for central — factors of 12.4 and 31 — and the ratio to dissemination is $2(p-1)/\lceil \log_2 p \rceil$, so it widens without bound. There is no regime in which a harness with agent ranks should choose this barrier.

There are two honest things to say in its favour and the sweep supports both. First, it is the cheapest barrier in the sweep by message count: 62 messages at $p = 32$ against dissemination's 160, because dissemination's reach must overshoot p. On this agent-free fabric that decides the outcome — linear spans 0.102 s at $p = 32$ against dissemination's 0.236 s, and accumulates 0.82 rank-seconds of receive-blocking against 4.30 — because with no invocation latency the per-message cost dominates and the round count is nearly free. The crossover is at a per-round cost of roughly $(160 - 62)/(62 - 5) \approx 1.7$ times the per-message cost; a fabric round trip is 0.003 s and an agent invocation is on the order of 10^0–10^1 s, so the crossover is exceeded by three orders of magnitude the moment a rank is filled by a model. This family is therefore a measurement of the $\alpha \to 0$ end of the axis, and the inversion it shows is real for that end and irrelevant to the other.

Second, and more usefully, it is the algorithm to reach for when the leaves are the thing you care about. A leaf pays two messages regardless of p; the whole $2(p-1)$ falls on the root. A harness with one privileged coordinator and $p - 1$ cheap workers, where the coordinator is going to serialise anyway because it is about to integrate everyone's output, loses little by serialising the barrier into the same rank. That is the shape of the integration phase in `real-sw-p8-full`, and it is why the pattern keeps getting hand-written.

The choice a harness should usually make instead is neither of these but `central`, which the sweep cannot measure. It costs two fabric round trips and zero messages, and it is the only barrier that can name the ranks that failed to arrive — so `barrier()` coerces `algorithm = "central"` whenever the policy is `PROCEED`, `SHRINK` or `REVOKE`. This family exists only because `bench_collectives` passes `policy=BarrierPolicy.WAIT`, which is the policy the docstring describes as "useful only when

debugging". The four policies AgentMPI adds to MPI are exercised in the `mb-free__faults-*` runs, and they are the reason the barrier was rethought at all: with rank 3 dead, `PROCEED` and `SHRINK` completed after 15.2 s having sent zero messages and having correctly reported `"absent": [3]`, whereas `WAIT` and `RAISE` left no `coll.barrier` record at all. Their traces show 17 sends against 10 receives — seven messages that were never delivered — with the last send recorded at 0.03 s and the seven surviving ranks then finalising together at 15.25 s, having emitted no completion event at all. Note that those runs exercise `dissemination`, not this algorithm: the message tags run `barrier:0:1:0` through `:2`, three rounds for $p = 8$, and `bench_faults` does not name an algorithm so it takes the default. A barrier that blocks forever is how agent harnesses hang, and `mb-free__faults-wait` is what that looks like in a trace: an unmatched message pattern and a missing collective record.

7 Threats to this reading

These runs contain no agents. No executor is recorded, the viewer's agent-call panel reads 0, and the 0.013–0.125 s wall times are SQLite writes and event-log appends. The consequence is sharper for `barrier` than for any other collective, because a barrier's cost is by construction the cost of its slowest participant, and thread start-up jitter is not heavy-tailed in the way agent latency is. Nothing in this family measures the quantity that governs barrier cost in the agent setting.

The place it is measured is the real-agent runs. In `real-sw-p8-full`, the `integrate-r1` barrier held its eight ranks for 0.00, 22.45, 60.72, 117.25, 140.02, 177.54, 191.30 and 215.56 seconds — 924.8 rank-seconds, a 9.6-fold spread between the fastest and slowest waiter — and `integrate-r2` for a further 428.3. Across that run's four barriers, 1,574 of the 1,666 rank-seconds spent in all collectives were barrier time, which is 94 % of collective blocking and 39 % of the run's total rank-time. That is the reading a barrier analysis needs, and it comes from there rather than here.

The per-rank wall times in this sweep are not arrival times. The `coll.barrier` records do report per-rank durations, and at $p = 32$ they range from 0.0112 s to 0.0823 s (a factor of 7.4). That quantity is the interval each rank spent inside the barrier, so the last rank to arrive reports nearly nothing and the first reports the whole span; the ratio is a measure of start-up jitter, not of a straggler distribution, and it should not be extrapolated.

The round-count defect is an omission whose consequences I cannot demonstrate. The evidence is direct — no assignment in the source, twenty-five assignments in the sibling branches, `"rounds": 0` in all forty-two sweep runs and all four real-run barriers — but no published result appears to depend on a barrier's round count, precisely because it has always been zero. The risk is that a calibration or selection step reading `rounds` would price a barrier at nothing. The related mismatch, between the implementation's "2 phases" and the closed form's "$2(p-1)$ serialised messages", would remain after the one-line fix and needs a decision about which quantity the field means.

Single samples. Thirteen distinct sizes, one or two runs each, nothing averaged. The integer counts are deterministic and the duplicated sizes confirm them; every wall-clock figure above is one observation.